

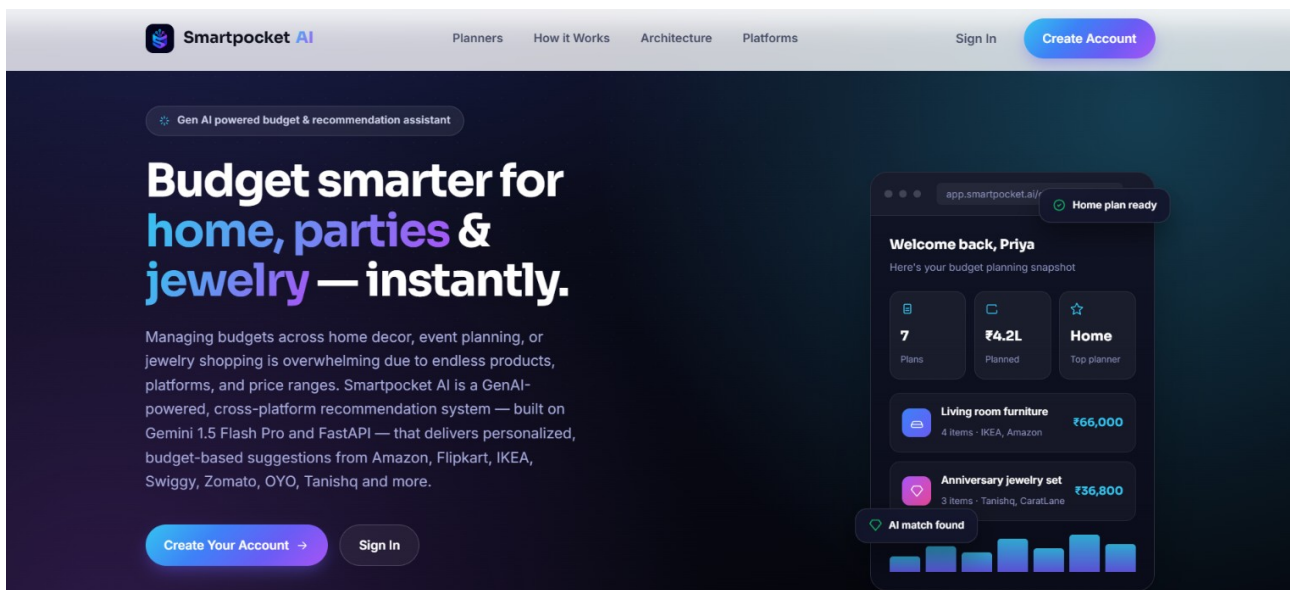


PHASE 1

Product Overview and Requirements

Abstract, problem definition, scope and requirements

SMARTPOCKET AI • SMART BUDGET AND RECOMMENDATION ASSISTANT



Project visual supplied with the source material

Abstract

SmartPocket AI is a web-based smart budget and recommendation assistant designed to help users create practical purchase plans within a fixed budget. The application provides three focused planning modules for home interiors, parties, and jewelry. Each module collects scenario-specific inputs and produces structured recommendations that include categories, quantities, estimated costs, remaining budget, and relevant shopping-search links.

The platform uses a responsive HTML, CSS, and JavaScript frontend with a FastAPI backend. User authentication is managed through JSON Web Tokens, passwords are protected using bcrypt hashing, and SQLite with SQLAlchemy stores user accounts and recommendation history. Google Gemini generates candidate recommendations, while backend validation and deterministic normalization recalculate totals, enforce the supplied budget, and reject incomplete or malformed responses.

The design separates presentation, API orchestration, AI integration, security, and persistence into maintainable modules. The system also supports private plan history, safe deletion based on ownership, environment-based secret management, responsive interfaces, and optional outfit-image input for jewelry recommendations.

The project demonstrates how generative AI can support personalized planning without treating model output as authoritative. By combining AI-generated suggestions with structured schemas, authentication, budget controls, and deterministic application logic, SmartPocket AI provides a practical foundation for a



secure and extensible recommendation platform.

Keywords: Smart budgeting, recommendation system, FastAPI, Gemini AI, JWT authentication, budget normalization, responsive web application

1. Product Overview

1.1 Product Definition

SmartPocket AI is a web application that creates personalized purchase plans within a fixed budget. It supports three focused planning experiences: home interiors, party planning, and jewelry selection. The system combines guided user inputs with an AI recommendation layer and deterministic backend validation.

1.2 Problem

Users often compare many products, vendors, and price ranges before deciding how to spend a limited amount. Conventional expense trackers record money after it is spent, while generic chat interfaces may return unstructured suggestions that are difficult to validate, save, or reuse.

1.3 Product Response

- Collect only the inputs needed for the selected planning scenario.
- Generate category-wise recommendations in a predictable JSON structure.
- Recalculate and normalize totals before displaying a plan.
- Keep each generated plan in a private user history.
- Support an optional outfit image for jewelry recommendations.
- Present results in Indian Rupees with practical shopping-search links.

1.4 Scope Boundary

Included: Budget planning, recommendations, authentication, plan history, responsive pages, structured results, and optional image-assisted jewelry matching.

Not included: Order placement, payment processing, inventory guarantees, live price guarantees, or direct control of third-party marketplaces.

2. Requirements

2.1 Functional Requirements

ID	Requirement	Acceptance signal
FR-01	Register and authenticate a user	A valid session token is returned after successful authentication.
FR-02	Generate a home interior plan	The response contains categorized items, quantities, estimated costs and links.
FR-03	Generate a party plan	The response includes service allocations and venue suggestions.
FR-04	Generate a jewelry plan	The response includes jewelry suggestions and optional outfit analysis.
FR-05	Enforce the supplied budget	The returned spent value is not greater than total_budget.
FR-06	Save and retrieve plans	The authenticated user can list and reopen saved results.



ID	Requirement	Acceptance signal
FR-07	Delete history safely	A user can delete only entries owned by that user.
FR-08	Display session information	The dashboard shows current authenticated-session details.

2.2 Quality Requirements

- **Security:** passwords are hashed and protected endpoints require authentication.
- **Reliability:** malformed AI output is rejected or normalized before use.
- **Usability:** forms and results remain readable across desktop and mobile layouts.
- **Maintainability:** planner, authentication, history, schema, and AI concerns remain separated.
- **Observability:** API errors use meaningful HTTP status codes and messages.
- **Configuration safety:** API keys and signing secrets are loaded from environment variables.